

1. (1) 由于广度优先搜索的正确性,

$$u.d = \delta(s, u) \text{ 定值}$$

$\therefore$ 与邻接表次序无关

(2) 22-3的图中, 若 x 先于 t 入队

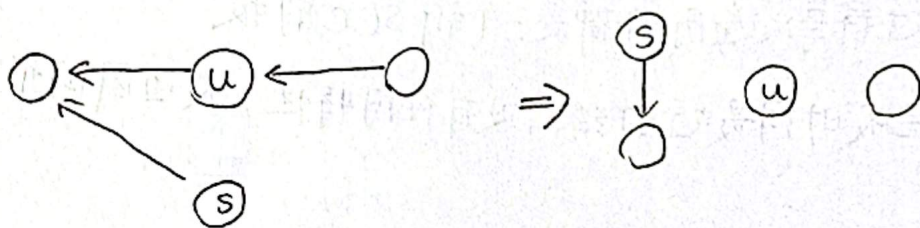
则会找到 u, y , t 处无点入队

得到的广度优先搜索树与原先不同

$\therefore$ 与邻接表次序有关

2. 使 u 能到达的结点均在访问 u 之前 被访问

~~且~~ 能到达 u 的结点均在访问 u 之后被访问



3. 使用邻接表 A , 设 $count=0$ 原理: 最小生成树

DFS - circle (G, s): (初始)

$s.color = gray$

$u = A[s]$

while $u \neq NULL$:

if $u.color \neq white$:

$u = u.next$

$count++$

continue

$u.color = gray$

$count++$

DFS - circle (G, u)

$u = u.next$ $\leftarrow$ return

if $count \geq |V|$:

judge = true 有环

return

n 结点

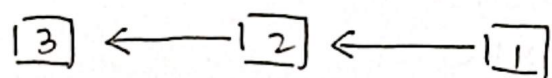
$n-1$ 条边

最多遍历 n 条边出结论

$\therefore O(V)$ 成立



4. 很明显不能



假设 x 为源点, y 第一个被遍历

则 $y.f$ 最小, 按该算法第一个开始

此时 SCC 只有一个, 显然不正确

原算法利用了 SCC 的根 $r.f$ 最大的特性

反转图帮助遍历时不会进入别的 SCC

这样每次遍历的都是一个新 SCC 的根

而完成时间最短的结点没有任何特性, 仅由图的形状和遍历顺序决定

